

Q.1 Write down the sequence of socket library functions invoked on the server side before a server is ready for communicating with the client using TCP. Give the important parameters and return values of these functions and write one sentence each about these functions.

Answer:

```
{  
  
  
  
  
  
  
}
```

1 mark for function in sequence

1 mark for all parameters

1 mark for explanation

- **Create a socket with the `socket()` function;**
- **Bind the socket to an address using the `bind()` function;**
- **Listen for connections with the `listen()` function;**
- **Accept a connection with the `accept()` function system call. This call typically blocks until a client connects with the server.**

`int socket(int family, int type, int protocol);`

Family specifies the protocol family (AF_INET , type is constant described by the type of socket(sock_stream, sock_dgram)

Returns 0 if success else -1

`int bind(int sockfd, const struct sockaddr *servaddr, socklen_t addrlen);`

Assigns a local protocol address to a socket IP+ 16 bit TCP port number.

Sockfd is the socket descriptor, servaddr is a pointer to a protocol-specific address, addrlen is the size of the address structure.

Returns 0 if success else -1

`int listen(int sockfd, int backlog);`

Listen function converts an unconnected socket into a passive socket, kernel should accept incoming connection requests directed to this socket.

Sockfd is the socket descriptor and backlog is the maximum number of connections the kernel should queue for this socket.

Returns 0 if success else -1

`int accept(int sockfd, struct sockaddr *cliaddr, socklen_t *addrlen);`

It is used to retrieve a connect request and convert that into a request.

Sockfd is a new file descriptor that is connected to the client that called connect().

Cliaddr and addrlen are used to return the protocol address of the client

Q. 2 Suppose within your Web browser you click on a link to obtain a Web page. The IP address for the associated URL is not cached in your local host, so a DNS lookup is necessary to obtain the IP address. Suppose that n DNS servers are visited before your host receives the IP address from DNS; the successive visits incur round trip times of $RTT_1, \dots, RTT_n$. Further suppose that the Web page associated with the link contains exactly one object, consisting of a small amount of HTML text. Let RTT_0 denote the RTT between the local host and the server containing the object. Assuming zero transmission time of the object, how much time elapses from when the client clicks on the link until the client receives the object?

Answer:

1] DNS queries sent over UDP so, each query takes RTT for the response to come.

Consider the IP address of total amount of time:

$$RTT_1 + RTT_2 + \dots + RTT_n$$

{2 marks : 1 mark [explanation] + 1 mark [answer]}

2] Server object request is HTTP call, which is sent over TCP.

1 RTT_o for connection request + 1 RTT_o for response = 2 * RTT_o

$$2RTT_o$$

{2 marks : 1 mark [explanation] + 1 mark [answer]}

Finally,

Time elapses from when the client clicks on the link until the client receives the object :

$$2RTT_o + RTT_1 + RTT_2 + \dots + RTT_n$$

Based on the answer, marks will be given in three categories : 0, 1 and 2.

Q. 3 A set of nodes form a connected graph where each edge denotes a neighbor relation. Two nodes with an edge between them are said to be one hop apart. Two hop neighbors are connected through an intermediate node

Design a protocol that permits a node to generate the list of all its 1-hop and 2-hop neighbors.

Clearly state the message structures and actions of all the entities involved [5]

Answer:

1st Method Using Broadcast:

1. Each node broadcasts a "1-hop request" message to all its neighbors with a TTL of 1.
2. When a node receives a "1-hop request" message, it responds with a "1-hop response" message and adds the sending node to its list of 1-hop neighbors. The "1-hop response" message has a TTL of 1.
3. The original node waits until it has received responses from all its neighbors, then sends a "2-hop request" message with a TTL of 2 to each of its 1-hop neighbors.
4. When a node receives a "2-hop request" message with a TTL of 2, it forwards the message to all its neighbors, except the node that sent the request.
5. When a node receives a "2-hop request" message with a TTL of 1, it responds with a "2-hop response" message and adds the sending node to its list of 2-hop neighbors.
6. The original node waits until it has received responses from all its 1-hop neighbors, then generates a list of all its 1-hop and 2-hop neighbors by examining its 1-hop and 2-hop neighbor lists.

Msg structure for source:

Request packet: [src addr, hop count = 2]

Response packet: [src addr = 1st neighbor, hop count = 1, dest = source node addr]

Msg structure for 1st neighbor:

Request packet: [src = 1st neighbor, hop count = 1]

Response packet: [src addr = 2nd neighbor source, hop count = 0, dest = 1st neighbor source addr]

Note: if any student write above steps combinly like only send 2-hop request and when intermediate node receive it. It will perform 2 task, 1st check if hop count is ≥ 0 then it is valid packet otherwise discard it, and it forward to all other node and also respond to the source node with id of node or address of node.

Mark distribution: [2+3 = 5]

2 [1 mark for idea and action, 1 mark for msg structure] marks if you write actions and msg structure for 1-hop neighbors.

3 [1 mark for msg structure + 1 mark for idea + 1 mark for action] marks if you write correctly all things for 2-hop neighbors like basic idea, actions and msg structure.

(OR) 2nd Method Using msg exchange:

1. **Message structure:** This message is used to communicate the routing table of a node to its neighbors. The message contains the ID of the sending node and its routing table, which consists of a list of destination nodes and hop count.

Msg structure for source:

Request packet: [src addr, hop count = 2]

Response packet: [src addr = 1st neighbor, hop count = 1, dest = source node add]

Msg structure for 1st neighbor:

Request packet: [src = 1st neighbor, hop count = 1]

Response packet: [src addr = 2nd neighbor source, hop count = 0, dest = 1st neighbor source addr]

2. **Actions of the initiating node:**

- a. Initialize the node's routing table to include all 1-hop neighbors and set the distance to each neighbor to 1.
- b. Broadcast a "distance-vector" message to all of its neighbors to share its routing table.
- c. Wait for responses from all neighbors.
- d. Update its routing table based on the received "distance-vector" messages, and set the distance to each 2-hop neighbor to 2.
- e. Generate a list of all 1-hop and 2-hop neighbors based on the updated routing table.

3. **Actions of intermediate nodes:**

- a. Receive a "distance-vector" message from a neighbor.
- b. Update the routing table based on the received message and propagate the updated table to its neighbors.

Above protocol run for 2 round to discover all 1-hop and 2-hop neighbor

Overall, the DVR protocol works by propagating routing information throughout the network, allowing each node to build a routing table that contains the distance to all other nodes in the network. By setting the initial distance to each 1-hop neighbor to 1 and the initial distance to each 2-hop neighbor to 2, we can easily generate the list of all 1-hop and 2-hop neighbors once the routing table has been constructed.

Mark distribution: [2+3 = 5]

2 [1 mark for idea and action, 1 mark for msg structure] marks if you write actions and msg structure for 1-hop neighbors.

3 [1 mark for msg structure + 1 mark for idea + 1 mark for action] marks if you write correctly all things for 2-hop neighbors like basic idea, actions and msg structure.

Question 3: A packet switch receives a packet and determines the outbound link to which the packet should be forwarded. When the packet arrives, one other packet is halfway done being transmitted on this outbound link and four other packets are waiting to be transmitted. Packets are transmitted in order of arrival.

Suppose all packets are 1,500 bytes and the link rate is 2 Mbps. What is the queuing delay for the packet?

Solution and Marking Scheme:

1] Time to transmit 1 packet (p):

$$p = (1500) * 8 / (2 * 10^6) \quad \text{or} \quad p = (1500) * 8 / (2^{20})$$

[0 or 1.5 marks for correct calculation]

2] Calculating Total Queuing delay T_q :

The delay that occurs when frames or cells wait in a device before being forwarded, and often a major component of latency.

These are 3 different approaches are fine for calculating total queuing delay other solutions will not be considered. And some explanation is expected for justifying the solution.

2.1 $T_q = 4 * p$ [1 marks]

2.2 $T_q = 4 * p + (1/2) * p$ [1.5 marks]

2.3 $T_q = 5 * p + (1/2) * p + \text{explanation}$ [1 marks]

[This is the general way of solving the question..]

QUEUING DELAY = (TOTAL NO. OF BITS NOT YET TRANSMITTED)/ RATE OF TRANSMISSION*(in bits)*

general formula: →

- length of each packet = L,
- the transmission rate = R,
- no. of bits of the currently-being-transmitted packet that has already been transmitted = x
- packets already in the queue = n

$$T_q = (n \cdot L + L - x) / R$$

The first packet is halfway through → half of its bits are already transmitted, therefore half of the bits of that packet are still in the queue.

packet size = 1500

transmitted bits (x) = $1500/2 = 750$

remaining bits (L-x) = $1500 - 750 = 750$

$$T_q = (4 \cdot (1500) + 750) \cdot 8 / (2 \cdot 10^6)$$

$$T_q = 27 \text{ msec} = 0.027 \text{ sec}$$